

OTA Diagnostic Tool - Web API and Frontend Interaction Specification Template

1. Document Metadata

- Project: OTA Diagnostic Tool
- Target: ESP8266EX
- Firmware version:
- Spec version:
- Author:
- Last updated:

2. Goals

- Serve a browser UI from the device.
- Ensure browser loads HTML, CSS, and JS as separate HTTP resources.
- Start frontend runtime logic after JS is received by the browser.
- Poll selected API endpoints periodically for live diagnostics.
- Support user-driven actions through event-triggered API calls.

3. Base URL and Transport

- Device host: http://
- API base path: /api
- Protocol: HTTP/1.1
- Data format for APIs: application/json
- Character encoding: UTF-8

4. Browser Resource Loading Contract

4.1 Static resources

- GET /
 - Purpose: Main HTML page
 - Content-Type: text/html

- GET /assets/app.css
 - Purpose: Stylesheet
 - Content-Type: text/css
- GET /assets/app.js
 - Purpose: Frontend runtime logic
 - Content-Type: application/javascript

4.2 Required browser behavior

1. Browser requests /.
2. Server returns index HTML.
3. Browser parses HTML and requests referenced CSS and JS files.
4. Browser receives and applies CSS.
5. Browser receives JS script and starts executing it.
6. JS initializes event handlers and periodic polling loops.

4.3 Caching policy (recommended)

- HTML: Cache-Control: no-cache
- JS/CSS: Cache-Control: max-age=, plus versioned URLs if possible
- API responses: Cache-Control: no-store

5. Frontend Runtime Behavior Contract

5.1 Startup sequence

- On JS start:
 - Build initial UI state.
 - Register event listeners for controls.
 - Perform one immediate diagnostics fetch.
 - Start periodic polling timers.

5.2 Polling policy

- Polling mode: fixed interval
- Default cycle time: 200 ms
- User configurable cycle time: yes
- Allowed cycle range (recommended): 200 ms to 5000 ms

- Apply behavior when cycle changes:
 - Stop existing timer.
 - Start new timer with updated interval.

5.3 Event-driven actions

- UI actions trigger on-demand API calls.
- Event actions do not wait for next polling tick.
- Polling remains active during event calls unless explicitly paused.

6. API Response Envelope (recommended)

All API responses should follow one structure.

- status: "ok" | "error"
- service: service identifier string
- ts_ms: device timestamp in milliseconds
- data: object (for successful responses)
- error: object (for failure responses)

6.1 Success response shape

- status: ok
- service: <service_name>
- ts_ms:
- data:

6.2 Error response shape

- status: error
- service: <service_name>
- ts_ms:
- error:
 - code: <short_code>
 - message: <human_readable>
 - details:

7. HTTP Status Code Policy

- 200 OK: Success

- 202 Accepted: Async job accepted (OTA start, self-test start)
- 400 Bad Request: Invalid input or malformed request
- 401 Unauthorized: Missing/invalid auth
- 403 Forbidden: Authenticated but not permitted
- 404 Not Found: Unknown route
- 409 Conflict: State conflict (OTA already running)
- 422 Unprocessable Entity: Input format valid but semantically invalid
- 500 Internal Server Error: Unexpected firmware error
- 503 Service Unavailable: Temporary subsystem unavailable

8. Service Catalog Template

Use one block per service.

Service Template

- Name:
- Endpoint: /api/
- Purpose:
- Trigger: polling | event-driven | startup-once
- Default frequency: <fill, if polling>
- Request params/body:
- Success status:
- Error status:
- Success data fields:
- Error codes:
- Notes:

9. Suggested Starter Services

9.1 Core and diagnostics

- GET /api/health
- GET /api/info
- GET /api/diag/summary

- GET /api/diag/memory
- GET /api/diag/errors

9.2 Network

- GET /api/network/status
- GET /api/network/scan
- POST /api/network/connect
- POST /api/network/disconnect

9.3 Control

- POST /api/control/restart
- POST /api/control/safe-mode

9.4 OTA

- GET /api/ota/status
- POST /api/ota/check
- POST /api/ota/start
- GET /api/ota/progress
- POST /api/ota/cancel

9.5 Logging

- GET /api/logs/recent
- POST /api/logs/clear

10. Polling Matrix Template

Service	Endpoint	Trigger	Interval (ms)	Timeout (ms)	Retry policy	Notes
Health	/api/health	polling	1000	500	exponential backoff	Liveness indicator
Summary	/api/diag/summary	polling	500	400	immediate retry x1	Main dashboard cards
Memory	/api/diag/memory	polling	2000	500	no retry	Lower priority
Errors	/api/diag/errors	polling	1000	500		

Service	Endpoint	Trigger	Interval (ms)	Timeout (ms)	Retry policy	Notes
					immediate retry x1	Active faults
Logs	/api/logs/recent	event-driven	n/a	800	manual refresh	Open log panel

11. Frontend Event Matrix Template

UI Control	Event	API Call	Method	Expected Result	UI Update
Restart button	click	/api/control/restart	POST	202 accepted	show reboot pending
Safe mode toggle	change	/api/control/safe-mode	POST	200 ok	update mode badge
Refresh cycle input	apply	local timer update	n/a	interval changed	show active interval
Wi-Fi connect form	submit	/api/network/connect	POST	200 or error	show connection state

12. Reliability and UX Rules

- If a polling call fails, show stale-data indicator and keep last good values.
- Cap concurrent inflight polling requests to avoid overload.
- Skip overlapping calls for the same endpoint when previous request not finished.
- Add per-endpoint timeout to avoid blocking update loop.
- Suspend high-frequency polling during OTA transfer.

13. Security Rules (minimum)

- Protect control and OTA endpoints with authentication.
- Never return secrets (passwords, tokens) in API responses.
- Validate all client inputs and ranges before applying.
- Rate-limit sensitive endpoints such as restart and OTA start.

14. Acceptance Checklist

- Browser loads / and renders base page.
- Browser requests CSS and JS as separate resources.
- JS starts automatically after script load.
- JS performs immediate first data fetch.
- JS starts periodic polling according to configured cycle time.
- User can change cycle time and polling interval updates correctly.
- API responses follow envelope and status conventions.
- Error states are visible and recover gracefully.

15. Versioning Notes

- Keep API paths stable.
- Add optional fields rather than breaking existing fields.
- If breaking changes are needed, introduce /api/v2.